LED 点滅実装検証報告書

作成日 2026 年 6 月 22 日
宇部工業高等専門学校 制御情報工学科 4 年 14 番
木村友亮

1. 実験方法

以下を使用し、機能検証/非機能検証を行う

- H8 実習キット(H8/3069F)
- デジタルオシロスコープ DCS1054B
- H8 実習キットエミュレータ

オシロスコープは ポート 4 からつながる LED に接続し、波形を観察する。
オシロスコープの自動測定機能を使用し、複数の波形が表示されている状態での周期(平均値)を測定結果として用いる。

2. 機能検証（定性的検証）

検証項目

- コマンドライン引数 `-B` 、 `-N` 、 `-F` 、 `-DEBUG1` が正しく動作すること。
- 指定した点灯パターン（ビットマスク）が正しく反映されること。
- 指定した点滅モード（0 〜 3）がそれぞれ仕様通りの挙動を示すこと。
- usage メッセージが仕様通りに表示されること。
- デバッグモード有効時の挙動。
- LED 機能の変更がモータなどの他機能に影響を与えないこと。

検証方法

- 各オプションを個別および組み合わせてプログラムを実行し、挙動を目視で確認する。
- コマンドラインから入力した引数とプログラムが表示する実行時オプション表示が一致することを確認する。
- デバッグモードをオンにした場合の数値 LED 表示の挙動を確認する。

- モータ制御など LED 以外の機能について、変更前（仕様 V0）と挙動が変わらないことを確認する。

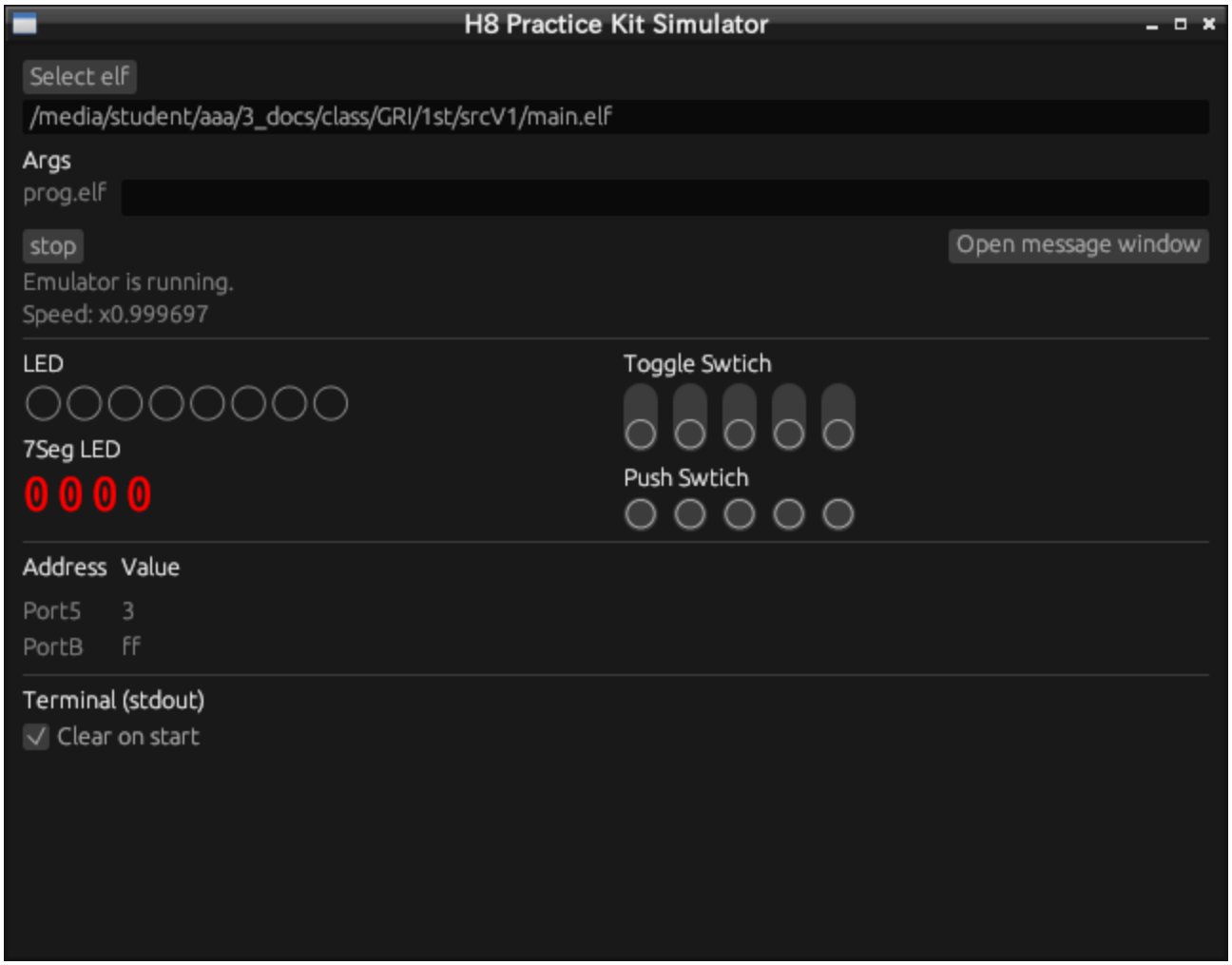
検証結果の概要

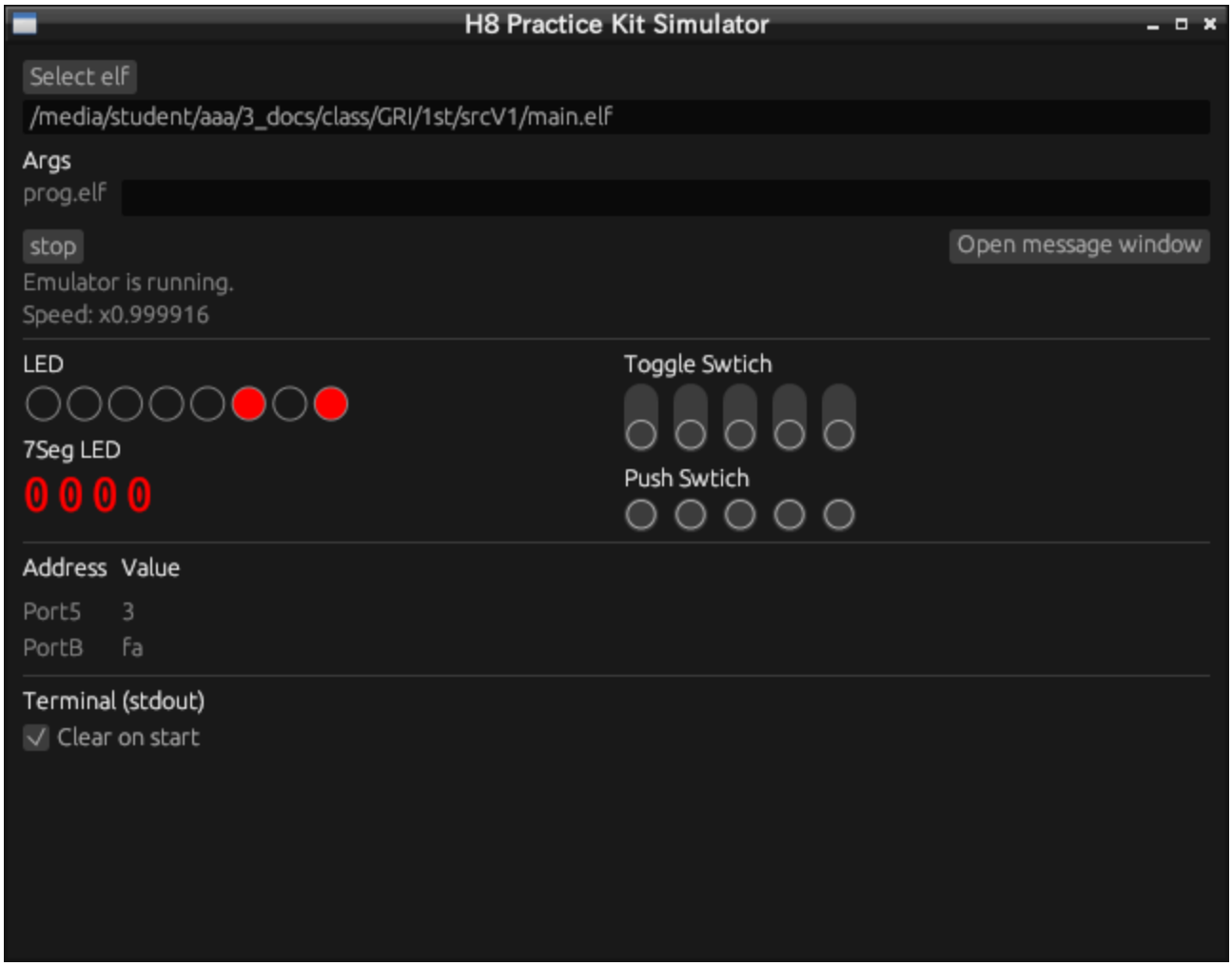
検証項目	テスト条件	期待結果	実際の結果	合否
点滅する LED パターンの指定(引数 N)	引数に-N0xee を指定(oooxooox)	ポート1,2,3,6,7が周期1000msで点滅	ポート1,2,3,6,7がおおよそ周期1000msで点滅	合格
点滅周期の指定(引数 B)	引数に-B500 を指定	ポート 0,2 が 400ms周期で点滅	ポート 0,2 が500msよりも短い周期で点滅	合格
点滅モードの指定(引数 F) モード0	引数に-F0 を指定	ポート 0,2 が点灯したまま消灯しない	ポート 0,2 が点灯したまま消灯しない	合格
	引数に-F1 -B599 を指定	ポート 0,2 が 400ms周期で点滅	ポート 0,2 がおおよそ0.5秒よりも短い周期で点滅	合格
	引数に-F1 -B600 を指定	ポート 0,2 が 600ms周期で点滅	ポート 0,2 がおおよそ0.5秒よりも長い周期で点滅	合格
点滅モードの指定(引数 F) モード2	引数に-F2 -B499 を指定	ポート 0,2 が 400ms周期で点滅	ポート 0,2 がおおよそ0.5秒よりも短い周期で点滅	合格
	引数に-F2 -B500 を指定	ポート 0,2 が 600ms周期で点滅	ポート 0,2 がおおよそ0.5秒よりも長い周期で点滅	合格
点滅モードの指定(引数 F) モード3	引数に-F3 -B400/-B500/-B600を指定	3つパターンがそれぞれ違う速度で点滅する	3つパターンがそれぞれ違う速度で点滅する	合格
デバッグターミナル出力(引数 DEBUG)	引数に-DEBUG1を指定	ターミナルにモーター回転数が出力される	大量の“motor_rotation_count = 0”がターミナルに出力された	合格
	引数に-DEBUG1を指定	数値LED表示が乱れる	乱れが観測された	合格
usageメッセージの表示	ボタンとトグルを操作し、 プログラムを停止させた後、 エミュレータのターミナルを確認する	ターミナルに以下が出力される ``` usage: ${実行ファイル名} [-Bxxx] [-Fy] [-Nzz] [xxx: LED blink cycle [msec] y: 1:blink, 0:nonblink zz: bit pattern -DEBUG1: turns on debug mode ```	シミュレータのボタンを操作して停止させたとき、 “usage:　y: 1:blink”と表示された。	不合格
usageメッセージの表示	ボタンとトグルを操作し、 プログラムを停止させた後、 ターミナルを確認する	ターミナルに以下が出力される ``` usage: ${実行ファイル名} [-Bxxx] [-Fy] [-Nzz] [xxx: LED blink cycle [msec] y: 1:blink, 0:nonblink zz: bit pattern -DEBUG1: turns on debug mode ```	改行で崩れたが、内容としては期待結果と同じものが表示された	合格

個別の検証結果

基準(引数なし)

```
prog.elf
```


H8 Practice Kit Simulator
Select elf
/media/student/aaa/3_docs/class/GRI/1st/srcV1/main.elf
Args
prog.elf
stop
Open message window
Emulator is running.
Speed: x0.999697
LED
7Seg LED
0 0 0 0
Toggle Swtich
Push Swtich
Address Value
PortS 3
PortB ff
Terminal (stdout)
Clear on start



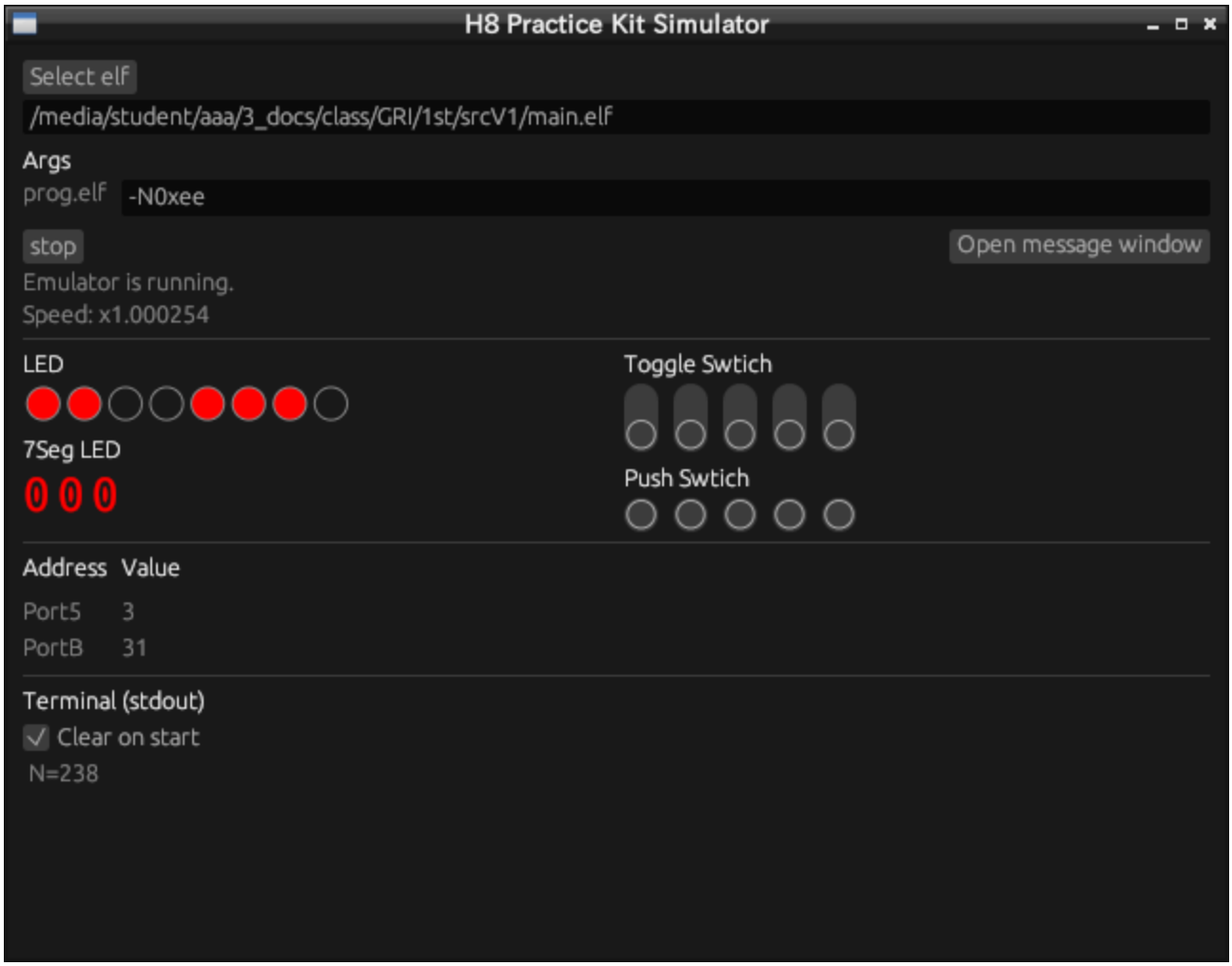

```
xxxxxoxo
xxxxxxxx
```

ポート 0・ポート 2 がおよそ 1 秒周期で点滅

点滅する LED パターンの指定(引数 N)

引数に-N0xee を指定(oooxooox)

```
prog.elf -N0xee
```




```
ooxxooox
xxxxxxxx
```

ポート 1,2,3,6,7 がおよそ 1 秒周期で点滅

点滅周期の指定(引数 B)

引数に-B500 を指定

```
prog.elf -B500
```

```
xxxxxoxo
xxxxxxxx
```

ポート 0,2 が およそ 0.5 秒より短い周期で点滅

点滅モードの指定(引数 F)

初期値 1000 は 200 の倍数であり、変化が期待できないため点滅周期 550ms(-B550) を指定する

1. 引数に-F0 を指定

   ```
   prog.elf -B550 -F0
   ```

   ```
   xxxxxoxo
   ```

   ポート 0,2 が点灯したまま消灯しない
2. 引数に-F1 を指定

   ```
   prog.elf -B550 -F1
   ```

   ```
   xxxxxoxo
   ```

   ポート 0,2 がおよそ 1 秒より短いが、0.5 秒よりは長い周期で点滅
   -B500 だけ指定したときと同じ挙動
3. 引数に-F2 を指定

   ```
   prog.elf -B550 -F2
   ```

   ```
   xxxxxoxo
   ```

   ポート 0,2 がおよそ 0.5 秒周期で点滅
   2 よりも点滅が速い
4. 引数に-F3 を指定

   ```
   prog.elf -B550 -F3
   ```

   ```
   xxxxxoxo
   ```

   ポート 0,2 がおよそ 0.5 秒よりも短い周期で点滅
   3 よりも点滅が速い

3. 非機能検証（定量的検証）

検証対象

- 点滅周期オプション `-B` で指定可能な時間間隔。
- 点滅モード `-F` による周期近似処理。

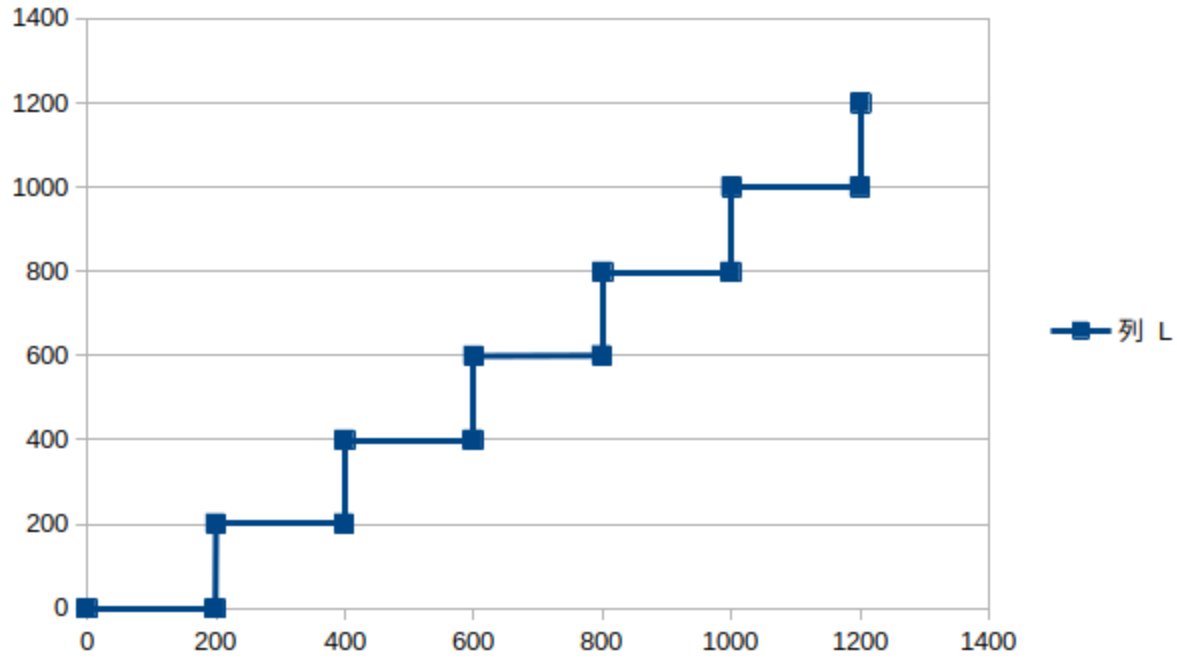
点滅モード

時間の関係で、点滅周期 `-F` を 1 に固定する。

テストケース一覧

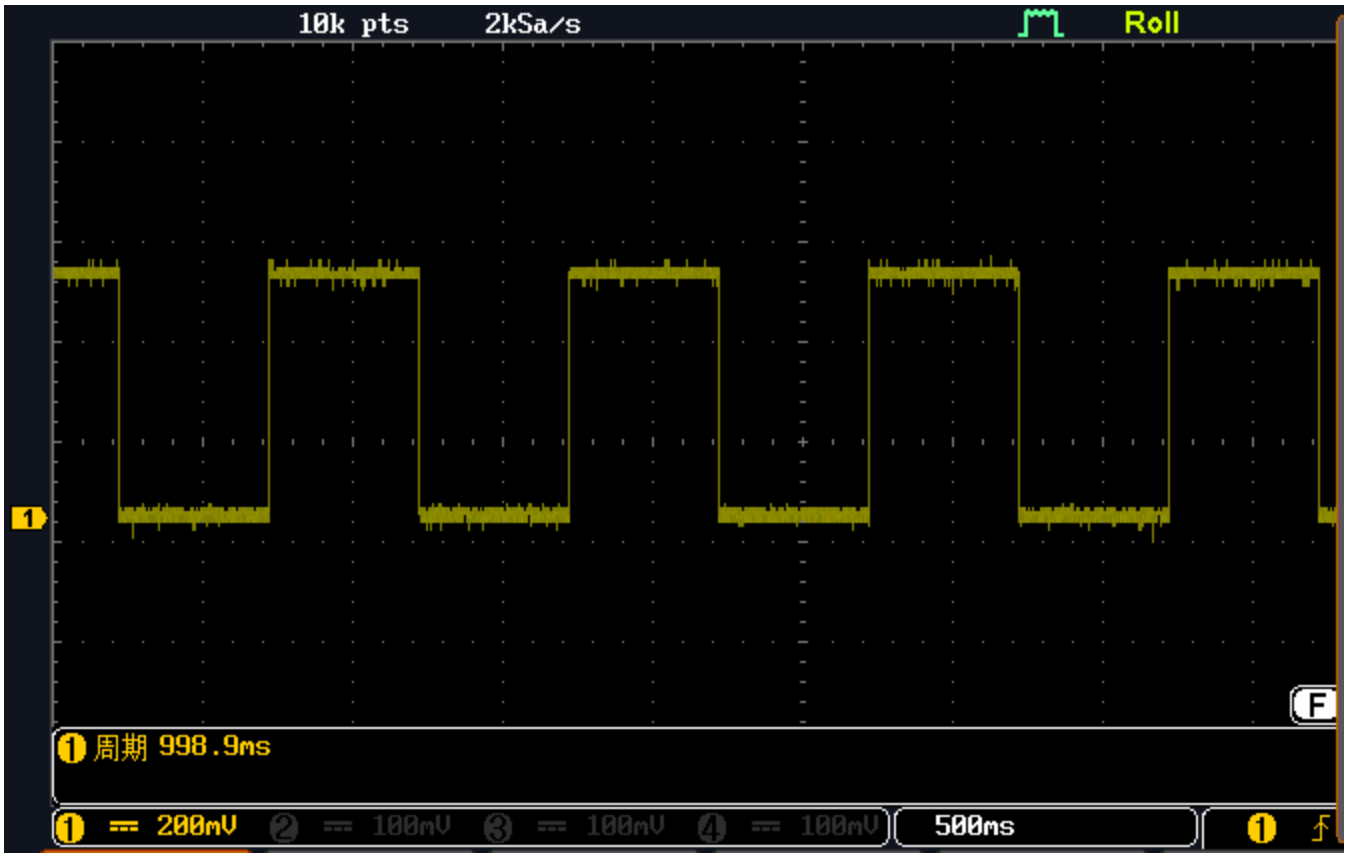
指定周期	モード	近似後の理想値	理論値	測定値
200	1	200	199.8848	200.5
399	1	200	199.8848	200.5
400	1	400	400.1792	399.5
599	1	400	400.1792	399.6
600	1	600	600.064	598.8
799	1	600	600.064	600
800	1	800	799.9488	798.4
999	1	800	799.9488	798.4
1000	1	1000	999.8336	999.8
1199	1	1000	999.8336	999.8
1200	1	1200	1200.128	1199.4

値の変化(モード 1)



測定波形

※後日改めて撮ったもののため、測定結果は検証結果表と差がある



4. 考察

- ## 機能検証で確認された問題点や特記事項

  少なくとも今回検証に使用したエミュレータとターミナルでは print 出力が不安定で、改行やスペースなどの処理はきれいに表示されない場合が多い

- ## 非機能検証の結果からわかる傾向や特記事項

  理論値/理想値と実測値は概ね一致している。
  指定周期そのものとの差は境界付近で大きくなる一方で、理論値との差は小さい。

- ## 理論値と測定値の差異に関する考察

  実測値と理論値の差は最大で約 1.3 ms であった。この差については、メインループでの出力更新タイミング、実習キットのクロック精度、オシロスコープの自動測定のばらつきなどが影響し

た可能性がある。ただし、今回の測定回数だけでは原因を特定できないため、各条件について複数回測定し、比較する必要がある。

- ## 全体的な評価と今後の改善提案

  概ね期待通りに動作した。
  改善が可能な点としては、機能検証をエミュレータのみで行い、LED 以外部分のモータ・7 セグなどの挙動については確認できていない点である。時間があればさらに検証項目を追加し、コードと照らし合わせて全体の機能を検証するべきだと考えられる。

付録：ソースコード

main.c

```c
/**
 * @file h8sample.c
 * @brief h8モーターを回転するプログラム
 * @author Yuji KATSUTA, Makoto TANABE
 * @date 2025.4.23
 */
#include "getint.h"
#include "timer.h"
#include "motor.h"
#include "led.h"
#include "nled.h"
#include "hardware_init.h"

int main(int argc, char **argv)
{
  // モーターとエンコーダーに関する変数
  int current_encoder_state;    // 現在のエンコーダーの状態
  int previous_encoder_state;   // 前回のエンコーダーの状態
  int motor_rotation_count = 0; // モーターの回転数
  int motor_on = 0;             // モーターの回転有無（0:停止、1:回転）
  // LEDに関する変数

  //===ここから===
  // int led_pattern = -1;        // LEDの点灯パターン（初期値：全点灯）
  int led_pattern = 0x05; // xxxxxoxo

  // LEDの点灯パターン
  int blink_mode = 1;
  //===ここまで

  int led_blink_period = 1000; // LEDの点滅周期（初期値：1000ms）
  // タイマーに関する変数
  int last_blink_interval = 0; // 0.5秒ごとにLEDを点滅させるためのカウンタ

  // デバッグモードに関する変数（オプション引数「-DEBUG」を読み込む）
  int debug = 0; // 初期値：デバッグモード無効

  // 引数の解析

  //===ここから===
  if (getint(argc - 1, argv + 1, "-N", &led_pattern) == 0)
  {
    printf(" N=%d ", led_pattern);
```

```
  printf(LF);
}
if (getint(argc - 1, argv + 1, "-F", &blink_mode) == 0)
{
  printf(" F=%d ", blink_mode);
  printf(LF);
}
//===ここまで===

if (getint(argc - 1, argv + 1, "-B", &led_blink_period) == 0)
{
  printf(" B=%d ", led_blink_period);
  printf(LF);
}
if (getint(argc - 1, argv + 1, "-DEBUG", &debug) == 0 && debug)
{ // オプション引数「-DEBUG」として0以外の値が指定された
  printf("DEBUG MODE");
  printf(LF);
}
hardware_init();

//===ここから===
ledblinkmode(blink_mode);
ledblinktime(led_blink_period);
//===ここまで===

init_timer0(COUNT_100MSEC, NULL);
start_timer0();
previous_encoder_state = motor_encoder_state();

while (1)
{
  if ((P5DR & 3) == 0)
  {
    break;
  } // スイッチが押されたら終了
  if ((P5DR & 1) == 0)
  {
    motor_on = 1;
  } // モーターON
  if ((P5DR & 2) == 0)
  {
    motor_on = 0;
```

```
} // モーターOFF
if (motor_on)
{
  if ((P5DR & 8) == 0)
  {
    motor_fwd();
  }
  else
  {
    motor_rev();
  }
}
else
{
  motor_stop();
}
current_encoder_state = motor_encoder_state();
if ((previous_encoder_state == 3) && (current_encoder_state == 1))
{
  motor_rotation_count++;
  if (motor_rotation_count > 9999)
  {
    motor_rotation_count = 0;
  }
}
if ((previous_encoder_state == 3) && (current_encoder_state == 2))
{
  motor_rotation_count--;
  if (motor_rotation_count < 0)
  {
    motor_rotation_count = 9999;
  }
}
previous_encoder_state = current_encoder_state;
number_led(motor_rotation_count);

//===ここから===
led(led_pattern);
ledexec();
/*
if ((timer_get_timerv0()->count / 5) != last_blink_interval)
{
  last_blink_interval = timer_get_timerv0()->count / 5;
```

```
      led_pattern = ~led_pattern;
      led(led_pattern);
    }
    */
    //===ここまで===

    if (debug)
    { // デバッグモードが有効のときはmcount値を標準出力する。
      printf("motor_rotation_count = %d", motor_rotation_count);
      printf(LF);
    }
  }
  motor_stop();
  stop_timer0();
  led(0);
  number_led_off();
  printf(" usage: %s [-Bxxx] [-Fy] [-Nzz] [-DEBUG1]" LF
         "   xxx: LED blink cycle [msec]" LF,
         "   y: 1:blink、0:nonblink" LF,
         "   zz: bit pattern" LF,
         "   -DEBUG1: turns on debug mode" LF,
         argv[0]);
  return 0;
}
```

led.h

```
/**
 * @file led.h
 * @brief LEDの制御モジュール(ヘッダー)
 * @author Yuji KATSUTA, Makoto TANABE
 */

#pragma once

/**
 * @brief LED点灯関数
 * @param [in] n 2進表示する数値
 * @details
 * 詳細説明
 * - LED点灯関数
 * - bit4が使用できないので、
 * - bit4を除いた7bitで2進数表示
 */

void ledout(int n);

void led(int pat);

void ledblinkmode(int mode);

void ledblinktime(int period);

void ledexec();
```

led.c

```c
/**
 * @file led.c
 * @brief LEDの制御モジュール
 * @author Yuji KATSUTA, Makoto TANABE
 */
#include "led.h"
#include "timer.h"

#ifndef H8_H
#define H8_H
#include <stdlib.h>
#include <h8/reg3067.h>
#include <mes2.h>
#endif /* H8_H */

typedef struct
{
	int pattern;
	int period;
	int mode;

} ledv;
typedef ledv *ledvp;
static ledv _ledValue;

ledvp lv(void)
{
	return &_ledValue;
}
void ledout(int n)
{
	// bit4は使用できない、bit4以外は1で点灯
	static int oled = 0;
	if (oled != n)
	{ // 同じ場合は実行しない。
		oled = n;
		PBDR = ~(((n & 0xf0) << 1) | (n & 0x0f)); // 2022.01.26 mod
	}
	return;
}

void led(int pat)
{
```

```
        _ledValue.pattern = pat;
        return;
}

void ledblinkmode(int mode) // led_blink_mode
{
        _ledValue.mode = mode;
        return;
}

void ledblinktime(int period) // led_blink_period
{
        _ledValue.period = period;
        return;
}

void ledexec()
{
        int count = timer_get_timerv0()->count;
        int period = _ledValue.period;
        int mode = _ledValue.mode;
        int is_blinking = 0; // false
        switch (mode)
        {
        case 0:
                is_blinking = 1; // 点灯
                break;
        case 1:
                if (count / (period / 200) % 2 == 0)
                        is_blinking = 1;
                break;
        case 2:
                if (count / ((period + 100) / 200) % 2 == 0)
                        is_blinking = 1;
                break;
        case 3:
                if (((count * 200) / period) % 2 == 0)
                        is_blinking = 1;
                break;
        default:
                is_blinking = 1;
                break;
        }
```

```
        if (is_blinking)
                ledout(_ledValue.pattern);
        else
                ledout(0);
}
```